CIVIQ – AI-POWERED CIVIC ISSUE REPORTING AND COMPLAINT MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted to the APJ Abdul Kalam Technological University

in partial fulfillment of requirements for the award of degree

Bachelor of Technology

in

Computer Science and Engineering

by

DEVIKA P A	**KNP23CS042**
ASWATHY S	**KNP23CS038**
AKSHARA N LEO MATHEW	**KNP23CS023**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COLLEGE OF ENGINEERING KARUNAGAPPALLY

KOLLAM, KERALA

October 2026

COLLEGE OF ENGINEERING KARUNAGAPPALLY

Thodiyoor – 690523

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision of Department

To be renowned centre for education, research, and innovation in the frontier areas of Computer Science and Engineering.

Mission of Department

- Establish and maintain an operational environment to acquire, impart, create and apply knowledge in Computer Science and Engineering and inter-disciplinary.
- Serve as a resource centre for innovation in design and development of hardware and software.
- Inculcate leadership qualities, professional ethics and a sense of social commitment.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COLLEGE OF ENGINEERING KARUNAGAPPALLY
2026–2027

CERTIFICATE

This is to certify that the report entitled **CIVIQ – AI-POWERED CIVIC ISSUE REPORTING AND COMPLAINT MANAGEMENT SYSTEM** submitted by **DEVIKA P A** (KNP23CS042), **ASWATHY S** (KNP23CS038), and **AKSHARA N LEO MATHEW** (KNP23CS023), to the Department of Computer Science and Engineering in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in **Computer Science and Engineering** of the APJ Abdul Kalam Technological University is a bonafide record of the project work carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Ms. Afrah Mehar
(Project Guide)
Assistant Professor
Dept. of CSE
CE, Karunagappally

Dr. Smitha P
(Project Coordinator)
Associate Professor and HOD
Dept. of CSE
CE, Karunagappally

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COLLEGE OF ENGINEERING KARUNAGAPPALLY-690523
2026–2027

CERTIFICATE

This is to certify that the report entitled **CIVIQ – AI-POWERED CIVIC ISSUE REPORTING AND COMPLAINT MANAGEMENT SYSTEM** submitted by **AKSHARA N LEO MATHEW** (KNP23CS023) to the APJ Abdul Kalam Technological University in partial fulfillment of the B.Tech. degree in Computer Science and Engineering is a bonafide record of the project work (CSD 416 Project Phase II) carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Ms. Afrah Mehar
(Project Guide)
Assistant Professor
Dept. of CSE
CE, Karunagappally

Dr. Smitha P
(Project Coordinator)
Associate Professor and HOD
Dept. of CSE
CE, Karunagappally

DECLARATION

We, **DEVIKA P A** (KNP23CS042), **ASWATHY S** (KNP23CS038), and **AKSHARA N LEO MATHEW** (KNP23CS023), hereby declare that the project report entitled **CIVIQ – AI-POWERED CIVIC ISSUE REPORTING AND COMPLAINT MANAGEMENT SYSTEM**, submitted for partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering of the APJ Abdul Kalam Technological University, Kerala, is a bonafide work done by us under the supervision of **Ms. Afrah Mehar** (Assistant Professor, Dept. of CSE, College of Engineering Karunagappally).

This submission represents our ideas in our own words, and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources.

We also declare that we have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data, idea, fact, or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the Institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not previously formed the basis for the award of any degree, diploma, or similar title of any other University or Institute.

DEVIKA P A (KNP23CS042)

ASWATHY S (KNP23CS038)

AKSHARA N LEO MATHEW

(KNP23CS023)

Karunagappally

28-10-2026

Abstract

Rapid urban expansion and aging public infrastructure pose significant operational challenges for urban local bodies (ULBs) and municipal corporations. Conventional civic grievance redressal mechanisms are fundamentally hindered by manual triage delays, error-prone inter-departmental dispatching, lack of verifiable audit trails, and the absence of citizen engagement channels. Consequently, critical municipal hazards—such as hazardous road potholes, unattended solid waste accumulations, water pipeline ruptures, and defective streetlights—often experience prolonged resolution latencies and lack public accountability.

To address these systemic municipal service delivery bottlenecks, this project presents **CiviQ** (**AI-Powered Civic Issue Reporting and Complaint Management System**), a full-stack, automated civic technology platform engineered for automated triage, intelligent spatial routing, and transparent grievance resolution. The core diagnostic intelligence integrates a computer vision pipeline utilizing deep Convolutional Neural Networks and **YOLOv8** feature extractors for multi-class civic hazard identification, generating precise bounding box spatial localizations and calibrated confidence scores (89%–97%) across five core defect taxonomies: road surface disintegrations (potholes), solid waste accumulations, water supply leaks, electrical luminaire faults, and transit pathway hazards.

To resolve jurisdictional routing ambiguities, CiviQ implements a **Shapely Point-in-Polygon (PIP)** geospatial engine that dynamically interrogates GeoJSON multi-polygons comprising all 55 administrative wards of the Kollam Municipal Corporation. Upon geofence boundary verification, the platform executes automated department dispatching across the Public Works Department (PWD), Corporation Health & Sanitation Wing, Kerala Water Authority (KWA), and Kerala State Electricity Board (KSEB). To foster participatory governance, a strict 1-upvote community consensus mechanism is enforced, automatically escalating complaints to High priority once reaching predefined civic thresholds ($\geq$ 3 upvotes) and triggering immediate citizen notifications. Every state transition (`Pending`, `Under Review`, `In Progress`, `Resolved`, `Rejected`) is immutably recorded within a normalized **MySQL 8.0** audit history alongside simulated cryptographic SHA-256 transaction hashes. Deployed on a lightweight, asynchronous **Flask** REST architecture with an interactive **Leaflet OpenStreetMap** frontend, CiviQ establishes an accountable, auditable, and low-latency digital bridge between citizens and municipal authorities.

Acknowledgement

We take this opportunity to express our deepest gratitude and sincere thanks to everyone who helped us to complete this project work and report successfully.

We express our sincere thanks to **Dr. Smitha Dharan**, Principal, College of Engineering Karunagappally, Kollam, and **Dr. Smitha P**, Head of Department, Computer Science and Engineering, College of Engineering Karunagappally, Kollam, for providing us with all the necessary infrastructural facilities, high-performance computing resources, and institutional encouragement.

We would like to express our profound sense of gratitude and indebtedness to our project guide **Ms. Afrah Mehar**, Assistant Professor, Computer Science and Engineering, College of Engineering Karunagappally, for her insightful guidance, constant encouragement, rigorous critiques, and invaluable mentorship throughout the entire lifecycle of this project.

We also express our sincere gratitude and indebtedness to our project coordinator **Dr. Smitha P**, Associate Professor and HOD, Computer Science and Engineering, College of Engineering Karunagappally, for her continuous coordination, constructive reviews, and valuable support during each phase of this project.

We also convey our sincere appreciation to all the faculty members and technical staff of the Computer Science and Engineering for their kind cooperation, constructive academic advice, and technical support during our design and development phases.

Finally, we express our wholehearted thanks to our parents, family members, and friends who provided unwavering moral support, patience, and encouragement that contributed immensely to the successful realization of this project.

DEVIKA P A (KNP23CS042)

ASWATHY S (KNP23CS038)

AKSHARA N LEO MATHEW (KNP23CS023)

Contents

Abstract i

Acknowledgement ii

List of Figures vi

List of Tables vii

List of Symbols and Abbreviations viii

1 Introduction 1

1.1 Background and Overview 2

1.2 Existing Municipal Grievance Systems and Their Limitations 3

1.3 Proposed Solution: The CiviQ Platform 3

1.4 Objectives of the Project 4

1.5 Scope and Operational Context 5

1.6 Organization of the Report 5

2 Literature Review 7

2.1 Base Paper: AI and Blockchain Architecture for Civic Governance . . 7

2.2 Smart City Grievance Redressal and Mobile Architectures 8

2.3 Computer Vision and Image Processing for Hazard Detection 8

2.4 Crowdsourced Collective Intelligence and Public Adoption 9

2.5 Summary of Identified Research Gaps 9

3 Problem Definition 11

3.1 Problem Statement 11

3.2 System Boundaries and Operational Constraints 12

3.3 Mathematical Formulations 12
3.3.1 Geospatial Point-in-Polygon Containment Formulation 12
3.3.2 AI Defect Classification and Bounding Box Regression 13
3.3.3 Dynamic Priority Escalation and SLA Formulation 14
3.3.4 Tamper-Evident Cryptographic Audit Hash Formulation . . . 14

4 Requirements Specification 15
4.1 User Classes and Characteristics 15
4.2 Functional Requirements (FR) 15
4.3 Non-Functional Requirements (NFR) 17
4.4 Hardware and Software Specifications 17

5 System Design 19
5.1 High-Level Architecture 19
5.2 System Workflow and Operational Pipeline 21
5.2.1 Proposed Methods Pipeline 21
5.2.2 Layered System Workflow 21
5.2.3 End-to-End System Flow Architecture 21
5.3 System Flowchart and Algorithms 22
5.3.1 System Flowchart 22
5.3.2 System Algorithm 23
5.3.3 Complaint Lifecycle State Transition Graph 27
5.4 Database Design and Schema Connectors 27
5.4.1 Relational Entity Schema Connectors 27
5.4.2 Level-1 Data Flow Architecture 29

6 Design and Methodology 30
6.1 AI Vision Triage and Geospatial Inference Pipeline 30
6.2 Geospatial Analysis with Kollam Municipal Ward GeoJSON 31
6.3 Automated Authority Routing Engine 32
6.4 Production RESTful API Architecture with Flask 33
6.5 Automated Integration Testing and Empirical Validation 33

7 Preliminary Analysis 35
7.1 Analysis of Computer Vision Models for Civic Hazard Triage 35

7.2 Analysis of Spatial Point-in-Polygon Algorithms 36
7.3 Analysis of Database Engines: Relational MySQL vs. NoSQL 36
7.4 Analysis of Authentication and Security Protocols 37

8 Feasibility Study **38**
8.1 Technical Feasibility 38
8.2 Economic Feasibility 38
8.3 Schedule and Resource Feasibility 39

9 Conclusion and Future Scope **41**
9.1 Conclusion 41
9.2 Limitations Encountered 42
9.3 Future Scope 42

References **43**

List of Figures

5.1 Detailed multi-tier system architecture of CiviQ showing Presentation, API Gateway, Business Logic/GIS, AI Triage, and Relational Persistence tiers. 20

5.2 Proposed operational methods pipeline of the CiviQ civic grievance redressal platform. 21

5.3 Layered system workflow and 6-stage execution lifecycle across diagnostic and operational layers. 21

5.4 Overall end-to-end system flow architecture from citizen upload to administrative redressal and audit logging. 22

5.5 System flowchart detailing grievance ingestion, MIME validation, AI vision triage, Shapely ward containment, DB insertion, upvote escalation, and administrative redressal. 23

5.6 Complaint lifecycle state transition and escalation DAG illustrating states, transitions, escalation pathways, and audit logging. 27

5.7 Relational entity and database schema connectors showing 7 normalized tables, primary/foreign keys, and relational cardinality. 28

5.8 Level-1 Data Flow Diagram (DFD) depicting external entities, processing stages, datastores, and notification feedback loops. 29

6.1 Staged AI vision triage and geospatial routing pipeline across the 6 operational stages. 30

8.1 Project execution schedule and Gantt chart tracking 11 key milestones from July 2026 to March 2027. 39

List of Tables

2.1 Comparative analysis of existing civic grievance platforms and CiviQ. 10

4.1 System Hardware and Software Configuration. 18

6.1 CiviQ Municipal Department Routing Rules. 32

6.2 Empirical 11-Step Automated Integration Test Suite Results. 34

7.1 Comparative analysis of candidate computer vision models for civic hazard detection. 35

List of Symbols and Abbreviations

Abbreviations

CNN	Convolutional Neural Network
YOLO	You Only Look Once (Object Detection Architecture)
GIS	Geographic Information System
PIP	Point in Polygon (Spatial Analysis)
GeoJSON	Geographic JavaScript Object Notation
ULB	Urban Local Body
LSGD	Local Self Government Department
PWD	Public Works Department
KMC	Kollam Municipal Corporation
KWA	Kerala Water Authority
KSEB	Kerala State Electricity Board
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
PBKDF2	Password-Based Key Derivation Function 2
SQL	Structured Query Language
ACID	Atomicity, Consistency, Isolation, Durability
SLA	Service Level Agreement
UI/UX	User Interface / User Experience
SPA	Single Page Application
OSM	OpenStreetMap
MIME	Multipurpose Internet Mail Extensions
UUID	Universally Unique Identifier
SHA-256	Secure Hash Algorithm 256-bit
SRS	System Requirements Specification
DFD	Data Flow Diagram
CORS	Cross-Origin Resource Sharing
RBAC	Role-Based Access Control

Mathematical and Computational Notations

$\mathbf{p} = (\phi, \lambda)$	Geographic coordinate tuple representing latitude ϕ and longitude λ
$\mathcal{W}_k$	Planar polygon geometry defining the spatial boundary of administrative ward k
$\text{Covers}(\mathcal{W}_k, \mathbf{p})$	Boolean spatial containment function checking if coordinate $\mathbf{p} \in \mathcal{W}_k$
$\mathcal{B} = (x, y, w, h)$	Spatial bounding box coordinates: origin offset (x, y), width w, height h
$\kappa \in [0.0, 1.0]$	Calibrated AI computer vision detection confidence metric
C	Set of classified civic hazard categories: {Pothole, Garbage, Water Leak, Streetlight, Fallen Tree}
$u_i \in \mathbb{N}_0$	Cumulative verified community upvote tally associated with complaint i
τ_{up}	Configurable civic escalation threshold ($\tau_{\text{up}} = 3$ upvotes)
τ_{hours}	Configurable civic escalation time window ($\tau_{\text{hours}} = 48$ hours)
$P(c_i)$	Discrete priority rating assigned to complaint: {Low, Medium, High, Critical}
H_{tx}	Hexadecimal simulated cryptographic hash verifying grievance state immutability
$\mathcal{A}_d$	Designated municipal authority department record mapped to complaint category
$\mathcal{S}$	Lifecycle grievance status: {Pending, Under Review, In Progress, Resolved, Rejected}

Chapter 1

Introduction

Rapid urbanization and expanding public infrastructure present unprecedented administrative and operational challenges for municipal corporations, urban local bodies (ULBs), and grama panchayats. Urban public services—encompassing transportation roadways, solid waste sanitation, clean drinking water distribution, nocturnal street lighting, and environmental safety—form the critical lifeblood of citizen well-being. However, the maintenance and timely repair of these sprawling infrastructure networks are persistently challenged by operational inefficiencies in civic grievance intake and redressal workflows.

Conventional municipal complaint systems remain heavily dependent on manual, fragmented, and bureaucratic reporting mechanisms. Citizens typically report grievances through physical visits to municipal ward offices, telephonic control rooms, or generic web portals that lack automated intelligence. This traditional paradigm suffers from severe systemic bottlenecks:

(i) **Manual Triage Latency and Deficit in Defect Verification:** Incoming grievances must be manually inspected, read, and classified by municipal operators who frequently lack photographic validation tools, resulting in prolonged initial triage queues.

(ii) **Jurisdictional and Departmental Routing Ambiguities:** Public infrastructure involves overlapping administrative authorities (e.g., Kollam Municipal Corporation, Public Works Department, Kerala Water Authority, and Kerala State Electricity Board). Manual operators frequently misroute complaints to incorrect departments, leading to jurisdictional ping-pong and administrative deadlock.

(iii) **Absence of Precise Geospatial Ward Discovery:** Complaints registered with vague text descriptions (e.g., "near temple road") prevent field inspection squads from locating defect sites accurately without repeated follow-up calls.

(iv) **Lack of Citizen Engagement and Community Consensus:** Conventional systems process grievances in administrative isolation without community input. Consequently, a severe crater on an arterial road affecting thousands of daily commuters receives the same operational priority as an isolated minor issue.

(v) **Absence of Verifiable Audit Trails and Transparency:** Citizens rarely receive transparent, tamper-evident updates regarding who reviewed their complaint, what remedial action was ordered, and when physical resolution was executed.

To directly address these acute municipal and technological challenges, this project presents **CiviQ**: an end-to-end, AI-powered civic issue reporting and complaint management platform. CiviQ bridges the operational gap between citizens and municipal field divisions by combining automated computer vision triage, Shapely Point-in-Polygon (PIP) geospatial boundary routing, crowdsourced community consensus with auto-escalation, and tamper-evident audit history logging.

1.1 Background and Overview

In contemporary smart governance, prompt redressal of urban defects is vital to safeguarding public safety, mitigating vehicle damage, preventing public health hazards, and optimizing municipal budgetary expenditure. Automated computer vision using Convolutional Neural Networks (CNNs) and real-time object detection models (such as YOLOv8) has achieved remarkable breakthroughs in real-world infrastructure inspection. Rather than relying on manual assessment, deep learning models can rapidly identify asphalt pavement craters, illegal garbage dumping, water pipeline ruptures, and defective streetlights directly from mobile camera captures.

Concurrently, Geographic Information Systems (GIS) and GeoJSON vector boundary polygons enable precise spatial localization. By executing Point-in-Polygon geometric algorithms, an incoming GPS coordinate can be deterministically matched to the exact administrative ward polygon, eliminating human guesswork and instantly identifying the designated ward councilor and zonal engineers.

1.2 Existing Municipal Grievance Systems and Their Limitations

Traditional municipal complaint platforms exhibit significant functional constraints:

- **Heuristic Portals without Automated Defect Classification:** Legacy web portals rely entirely on self-selected citizen dropdown categories, which are often erroneous or ambiguous.
- **Lack of Automated Department Cross-Matching:** Most existing systems do not dynamically map ward boundaries to departmental jurisdictions, causing delays when state-level agencies (such as PWD or KWA) operate within corporation municipal limits.
- **Opaque State Transitions without Cryptographic Verification:** Status updates such as "In Progress" or "Resolved" are frequently marked administratively without tamper-evident audit logs or citizen photographic verification.
- **Zero Civic Prioritization:** Single-user reporting mechanisms do not allow community co-residents to endorse or upvote localized issues to signal collective urgency.

1.3 Proposed Solution: The CiviQ Platform

The proposed CiviQ platform establishes an integrated, automated municipal resolution pipeline:

1. **Automated Computer Vision Triage:** Digital photographs uploaded by citizens are ingested by an AI inference service that evaluates visual features, classifies the civic defect into standardized categories (Pothole, Garbage Pileup, Water Leak, Streetlight Fault, or Pathway Hazard), detects bounding box coordinates, and outputs calibrated confidence scores (89%–97%).
2. **Shapely Point-in-Polygon Geospatial Engine:** The system extracts raw GPS coordinates and queries GeoJSON boundary multi-polygons comprising all 55 administrative wards of Kollam Municipal Corporation. The system automatically identifies the exact ward number and ward name (e.g., Ward 15 Asramam).

3. **Automated Multi-Departmental Dispatch:** Based on the detected defect taxonomy and identified ward, CiviQ dispatches the complaint to the appropriate authority (PWD Road Infrastructure, Corporation Health & Sanitation Wing, Kerala Water Authority, or KSEB Electrical Division).

4. **1-Upvote Community Consensus and Dynamic Escalation:** Community residents can view grievances on an interactive Leaflet OpenStreetMap and cast verified upvotes. To prevent vote manipulation, strict unique constraints enforce one upvote per user per complaint. When cumulative upvotes reach a configurable threshold (≥ 3), the complaint is automatically elevated to High priority with real-time alerts.

5. **Tamper-Evident Relational Audit History:** All lifecycle transitions (`Pending`, `Under Review`, `In Progress`, `Resolved`, `Rejected`) are permanently logged in a dedicated MySQL audit table with officer remarks, timestamps, and simulated cryptographic transaction hashes.

1.4 Objectives of the Project

The primary engineering, algorithmic, and operational objectives of this project are:

1. To architect and develop a robust, responsive web and mobile-accessible Single-Page Application (SPA) enabling citizens to report civic grievances with photos and GPS coordinates.

2. To engineer an AI Computer Vision triage endpoint capable of multi-class civic hazard classification and bounding box spatial regression with high confidence.

3. To implement a geospatial Point-in-Polygon (PIP) mapping engine using Python Shapely to resolve GPS coordinates to the 55 administrative wards of Kollam Municipal Corporation.

4. To design an automated multi-departmental dispatching module cross-matching defect categories and ward jurisdictions to PWD, Corporation Sanitation, KWA, and KSEB.

5. To build a crowdsourced civic engagement mechanism enforcing a strict 1-upvote constraint with automated SLA and priority escalation triggers.

6. To construct an administrative dashboard providing operational analytics, category breakdown percentages, departmental SLA performance tracking, and CSV export.

7. To establish a tamper-evident audit history mechanism logging state changes, timestamps, and simulated blockchain cryptographic hashes.

8. To validate the entire architecture through automated end-to-end integration test suites verifying authentication, reporting, routing, upvoting, and administrative workflows.

1.5 Scope and Operational Context

The platform is developed and evaluated with reference to the urban landscape of Kollam Municipal Corporation, utilizing actual ward boundary GeoJSON multi-polygons covering all 55 municipal wards. The software architecture is engineered to be horizontally extensible to other municipalities, corporations, and rural local self-governments (LSGD) across Kerala and India.

1.6 Organization of the Report

The remainder of this report is organized as follows:

- **Chapter 2 (Literature Review):** Reviews research in municipal e-governance, computer vision defect detection, GIS spatial analysis, crowdsourced civic computing, and web microservice architectures.

- **Chapter 3 (Problem Definition):** Formulates the civic grievance redressal challenge, defines operational constraints, and formalizes mathematical equations for spatial containment, defect regression, and priority escalation.

- **Chapter 4 (Requirements Specification):** Details the formal System Requirements Specification (SRS), user characteristics, functional requirements (FR1–FR12), non-functional requirements, and hardware/software environments.

- **Chapter 5 (System Design):** Details the multi-tier architecture, methods pipeline, 6-stage lifecycle workflow, end-to-end data flow, flowchart, the 12-stage operational algorithm, lifecycle state machine DAG, 7-table relational schema, and Level-1 DFD.

- **Chapter 6 (Design and Methodology):** Expounds the technical implementation of computer vision triage, Shapely geospatial analysis, authority routing algorithms, civic upvote consensus, REST API endpoints, and empirical integration testing.

- **Chapter 7 (Preliminary Analysis):** Delivers comparative technical evaluations of software architectures, computer vision frameworks, spatial indexing techniques, database engines, and authentication security.

- **Chapter 8 (Feasibility Study):** Evaluates technical viability, economic feasibility, and project schedule feasibility, accompanied by the 11-milestone project Gantt chart.

- **Chapter 9 (Conclusion and Future Scope):** Summarizes major project contributions, discusses operational limitations, and charts prospective research enhancements.

Chapter 2

Literature Review

Civic issue reporting and municipal complaint management sit at the intersection of urban informatics, artificial intelligence, geographic information systems, crowdsourced collective intelligence, and distributed web architectures. This chapter synthesizes recent advancements, critical methodologies, and benchmark systems established across contemporary academic literature.

2.1 Base Paper: AI and Blockchain Architecture for Civic Governance

The foundational architecture and operational paradigm for this project is established by the base paper by Prathyush et al. [1], titled *"CivicEye: AI and Blockchain-Powered Platform for Transparent Civic Issue Reporting and Resolution"*. Prathyush et al. introduced a dual-pillar framework that combines deep learning computer vision for automated civic grievance classification with blockchain-inspired cryptographic transparency to eliminate administrative tampering. The CivicEye framework addresses core structural vulnerabilities in municipal operations—specifically, subjective defect reporting, miscategorization, and lack of citizen visibility into resolution lifecycles. Building upon this foundational base paper, the CiviQ system extends these core concepts by integrating real-time spatial Point-in-Polygon GIS ward boundary routing, dynamic multi-departmental dispatching, and community upvote-driven SLA escalation while upholding the tamper-evident cryptographic provenance established in the base work.

2.2 Smart City Grievance Redressal and Mobile Architectures

A significant volume of contemporary literature investigates digital architectures for citizen reporting in smart municipal environments. Shama et al. [2] developed *CitySolution*, a mobile application utilizing deep learning for complaint task distribution across city corporation divisions, underscoring the importance of automated ticket routing to alleviate municipal backlogs. Similarly, Aakash et al. [3] proposed *CivicSense*, which leverages Natural Language Processing and Generative AI to identify, rank, and summarize citizen complaints, converting unstructured public grievances into actionable department-level insights.

Akaash Raj et al. [9] engineered *CIVORA*, an AI-based civic complaint registration system designed to streamline citizen submissions and optimize departmental responsiveness. Gavhane et al. [14] introduced *CityPluse*, a smart city issue reporting portal providing transparent monitoring dashboards for municipal tracking. Ali [15] presented *NagarMitram*, an integrated smart city platform focusing on bridging the communication gap between citizens and municipal engineers. Earlier work by Prasad et al. [18] established early foundations for civic complaint mobile apps under smart city missions, while Suvetha et al. [7] demonstrated web-based complaint management workflows for public authorities.

2.3 Computer Vision and Image Processing for Hazard Detection

The automated identification of civic infrastructure failures from imagery is a critical pillar of intelligent municipal triage. Early work by Danti et al. [4] utilized image processing techniques, Hough transforms, and color segmentation for detecting road potholes, faded lane markings, and road signs under Indian driving conditions. Nienaber et al. [11] demonstrated pothole detection using morphological filtering and edge detection on real-world vehicle footage, establishing baseline methods for road anomaly extraction.

Building on these image processing principles, Nimase et al. [8] introduced an

automated public complaint sorting model using computer vision, routing uploaded issue photographs to appropriate municipal wings. Harini et al. [6] proposed an AI-powered citizen evidence verification framework to validate incoming image submissions, filtering invalid or fraudulent reports before municipal dispatch. Suganthi et al. [5] designed an AI-based smart civic complaint system categorizing multi-domain municipal defects, including solid waste overflow, water leakages, and road surface damage.

2.4 Crowdsourced Collective Intelligence and Public Adoption

The operational efficacy of civic tech platforms depends on citizen participation and community consensus. Hou et al. [16] conducted an extensive empirical study on citizen acceptance and adoption factors for smart city mobile applications, showing that perceived transparency, procedural responsiveness, and ease of use govern sustained public participation. Gumalwad [13] explored crowdsourced civic issue reporting and resolution frameworks, demonstrating how collective neighborhood reporting uncovers localized urgency signals.

Shukla et al. [10, 17] analyzed civic complaint registration applications tailored for both rural and urban demographics, emphasizing the need for intuitive user interfaces, automated grievance tracking, and transparent status dissemination to build citizen trust across diverse demographic strata. Walwadkar et al. [12] investigated smart civic issue reporting architectures incorporating multi-department assignment and community feedback mechanisms.

2.5 Summary of Identified Research Gaps

Table 2.1 presents a comparative synthesis of existing civic reporting systems and frameworks investigated in the literature against the proposed CiviQ platform.

Table 2.1: Comparative analysis of existing civic grievance platforms and CiviQ.

System / Framework	**AI Defect Triage**	**GIS Ward Routing**	**Community Upvoting**	**Audit Trail Mechanism**	**Primary Focus / Operational Limitations**
CivicEye (Base Paper) [1]	CNN / AI Vision	Coordinate Pin	Status Tracking	Blockchain Ledger	Foundational base paper; pioneers AI + blockchain audit; lacks dynamic GIS ward polygon routing.
CitySolution [2]	Mobile Deep Learning	Text / GPS	Department Task	Firebase Auth Log	Task distribution mobile app; focused on city corporation task dispatching without spatial ward containment.
CivicSense [3]	Generative AI / NLP	Ward Text	Trend Analytics	Internal Server DB	Specializes in text-based NLP grievance summarization; lacks integrated real-time visual defect detection.
CIVORA [9]	Machine Learning	Manual Location	Tracking Portal	Standard DB Log	Automated registration portal; lacks crowdsourced community upvoting and cryptographic audit logging.
CityPluse [14]	None (Form-based)	Coordinate Pin	Issue Listing	Application DB	Issue reporting portal; lacks automated computer vision defect classification and spatial polygon GIS engine.
CiviQ (This Project)	**YOLOv8 / CNN**	**Shapely PIP (55 Wards)**	**1-Upvote Escalation**	**Crypto Audit Hash**	**Unified platform integrating AI triage, 55-ward spatial PIP routing, community upvoting, and tamper-evident audit.**

Chapter 3

Problem Definition

This chapter formalizes the operational and technological problem addressed by the CiviQ platform, defines system boundaries and constraints, and establishes the mathematical models governing computer vision defect triage, spatial containment, and civic escalation.

3.1 Problem Statement

Urban municipalities and local self-governments operate within fragmented, siloed departments, including Road Infrastructure (PWD), Solid Waste Management (Municipal Corporation), Drinking Water and Drainage (Water Authority), and Electrical Distribution (State Electricity Board). When civic defects occur in the public domain, the reporting and remediation process encounters multiple failures:

1. **Input Ambiguity and Validation Void:** Citizens often report issues with subjective descriptions, inaccurate categories, or poor-quality photos, requiring substantial human labor to parse and verify.

2. **Jurisdictional Ping-Pong:** Because municipal corporation borders encompass both municipal roads and state highways, manual operators regularly misassign grievances, delaying field response by weeks.

3. **Absence of Democratic Urgency Signals:** Traditional systems process complaints chronologically, preventing community co-residents from signaling collective severity or endorsing urgent neighborhood hazards.

4. **Lack of Verifiable Provenance:** Citizens are unable to verify whether municipal authorities conducted genuine inspections or arbitrarily marked complaints as resolved without remediation.

The objective of CiviQ is to eliminate these bottlenecks by engineering a unified platform that automatically classifies defect photos via computer vision, resolves administrative ward boundaries via spatial Point-in-Polygon mapping, dynamically dispatches tasks to responsible departments, enforces community upvote consensus, and maintains tamper-evident audit history records.

3.2 System Boundaries and Operational Constraints

The operational boundaries of the CiviQ system are defined as follows:

- **Geographic Boundary:** The primary spatial domain encompasses the 55 administrative wards of Kollam Municipal Corporation, Kerala, defined by GeoJSON boundary multi-polygons.
- **Input Constraints:** Image uploads are restricted to standard photographic MIME formats (`image/jpeg`, `image/png`, `image/webp`) with a strict file size ceiling of 10 MB. Geographic inputs must supply valid decimal latitude $\phi \in [-90, +90]$ and longitude $\lambda \in [-180, +180]$.
- **Authentication Constraints:** User passwords must be securely hashed using PBKDF2 or scrypt. Session security enforces URLSafeTimedSerializer JWT tokens valid for 7 days.
- **Civic Integrity Constraints:** A strict database-level unique constraint enforces exactly one upvote per citizen per complaint, preventing sybil or repeat vote inflation.

3.3 Mathematical Formulations

3.3.1 Geospatial Point-in-Polygon Containment Formulation

Let $\mathbf{p} = (\phi, \lambda)$ denote a geographic coordinate point where ϕ is latitude and λ is longitude. In GeoJSON standard formatting, planar coordinates are represented in

Cartesian order $(x, y) = (\lambda, \phi)$. Let $\mathcal{W} = \{\mathcal{W}_1, \mathcal{W}_2, \ldots, \mathcal{W}_{55}\}$ denote the set of administrative ward boundary polygons defining the municipal jurisdiction.

Each ward polygon $\mathcal{W}_k$ is defined by a sequence of exterior boundary vertices:

$$\mathcal{W}_k = \text{Polygon}([(\lambda_1, \phi_1), (\lambda_2, \phi_2), \ldots, (\lambda_n, \phi_n), (\lambda_1, \phi_1)]) \tag{3.1}$$

The spatial containment predicate $\text{Covers}(\mathcal{W}_k, \mathbf{p})$ is evaluated using Jordan's Curve Theorem via the ray-casting intersection test:

$$\text{Covers}(\mathcal{W}_k, \mathbf{p}) = \begin{cases} 1, & \text{if } \sum_{i=1}^{n} \mathbf{1}_{\text{intersect}}(R(\mathbf{p}), \overline{\mathbf{v}_i\mathbf{v}_{i+1}}) \equiv 1 \pmod 2 \\ 0, & \text{otherwise} \end{cases} \tag{3.2}$$

where $R(\mathbf{p})$ represents a semi-infinite horizontal test ray cast from point $\mathbf{p}$ along the positive abscissa, and $\overline{\mathbf{v}_i\mathbf{v}_{i+1}}$ represents boundary segment edges of polygon $\mathcal{W}_k$.

The resolved ward assignment function $\Omega(\mathbf{p})$ is uniquely determined as:

$$\Omega(\mathbf{p}) = \begin{cases} \mathcal{W}_k, & \text{if } \exists!\, \mathcal{W}_k \in \mathcal{W} \text{ such that } \text{Covers}(\mathcal{W}_k, \mathbf{p}) = 1 \\ \emptyset, & \text{if } \forall \mathcal{W}_k \in \mathcal{W},\ \text{Covers}(\mathcal{W}_k, \mathbf{p}) = 0 \quad \text{(Outside Jurisdiction)} \end{cases} \tag{3.3}$$

3.3.2 AI Defect Classification and Bounding Box Regression

Let $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$ denote an ingested civic defect image. The deep computer vision model extracts spatial feature representations and computes multi-class defect probabilities $\hat{\mathbf{y}}$ and bounding box coordinates $\hat{\mathcal{B}}$:

$$\hat{\mathbf{y}} = \text{Softmax}(f_{\theta}(\mathbf{I})) \in [0, 1]^{|C|} \tag{3.4}$$

where $C = \{\text{Pothole, Garbage, Water Leak, Streetlight, Tree Hazard}\}$.

The predicted category c^* and model confidence score κ are given by:

$$c^* = \arg\max_{c \in C} \hat{y}_c, \qquad \kappa = \max_{c \in C} \hat{y}_c \tag{3.5}$$

The spatial bounding box coordinates $\hat{\mathcal{B}}$ delineate the normalized localized hazard

boundaries within the image frame:

$$\hat{\mathcal{B}} = [x_{\text{top}}, y_{\text{left}}, w_{\text{box}}, h_{\text{box}}] \in [0, 100]^4 \tag{3.6}$$

3.3.3 Dynamic Priority Escalation and SLA Formulation

Let c_i denote a registered complaint record. The initial priority $P(c_i)$ is set to Medium upon registration. Let $u_i \in \mathbb{N}_0$ denote the total verified community upvotes recorded for complaint c_i, and let $\Delta t = t_{\text{current}} - t_{\text{created}}$ denote the elapsed operational duration since registration.

The dynamic priority escalation function $\mathcal{E}(c_i)$ is formulated as:

$$\mathcal{E}(c_i) = \begin{cases} \text{Critical}, & \text{if } u_i \geq 2 \cdot \tau_{\text{up}} \vee \Delta t \geq 2 \cdot \tau_{\text{hours}} \\ \text{High}, & \text{if } (u_i \geq \tau_{\text{up}} \vee \Delta t \geq \tau_{\text{hours}}) \wedge P(c_i) \in \{\text{Low}, \text{Medium}\} \\ P(c_i), & \text{otherwise} \end{cases} \tag{3.7}$$

where $\tau_{\text{up}} = 3$ upvotes and $\tau_{\text{hours}} = 48$ hours represent default configurable municipal policy thresholds.

3.3.4 Tamper-Evident Cryptographic Audit Hash Formulation

To ensure state immutability, every status change event $e_j = (c_i, \mathcal{S}_{\text{old}}, \mathcal{S}_{\text{new}}, u_{\text{officer}}, t)$ generates a cryptographic audit hash:

$$H_{\text{tx}} = \text{SHA256}\Big(\text{Code}(c_i) \,\|\, \mathcal{S}_{\text{new}} \,\|\, u_{\text{officer}} \,\|\, \text{Remarks} \,\|\, t \,\|\, H_{\text{prev}}\Big) \tag{3.8}$$

where $\|$ denotes string concatenation and H_{prev} represents the preceding state hash in the complaint's history sequence, guaranteeing cryptographic provenance.

Chapter 4

Requirements Specification

This chapter details the formal System Requirements Specification (SRS) for the CiviQ platform, specifying user characteristics, functional requirements, non-functional quality attributes, and the operational software and hardware stack.

4.1 User Classes and Characteristics

The system serves three primary user classes with distinct operational responsibilities:

1. **Citizens / General Public:** Residents of municipal wards who report civic defects, view local grievances on the interactive map, track complaint progress, upvote urgent community issues, and participate in discussion threads.

2. **Municipal Department Administrators / Field Officers:** Authorized municipal officials representing PWD, Health and Sanitation, Water Authority, and Electricity divisions. They inspect assigned complaints, update lifecycle statuses (`Under Review`, `In Progress`, `Resolved`, `Rejected`), provide mandatory resolution remarks, and monitor departmental SLA resolution rates.

3. **System Administrators:** Super-administrators responsible for managing ward boundaries, maintaining department configurations, monitoring server health, and auditing database integrity.

4.2 Functional Requirements (FR)

The functional requirements governing CiviQ are structured into twelve core modules:

- **FR1 (User Authentication and Security):** The system shall provide citizen registration and login endpoints utilizing secure PBKDF2/scrypt password hashing and signed URLSafeTimedSerializer JWT authentication tokens.
- **FR2 (Evidence Media Ingestion):** The system shall support image uploads via multipart form data, validating MIME headers against `image/jpeg`, `image/png`, and `image/webp`, enforcing a maximum payload size of 10 MB.
- **FR3 (Interactive Geolocation Pinning):** The system shall provide an interactive Leaflet OpenStreetMap interface allowing citizens to automatically detect device GPS coordinates or manually drop a pin to capture latitude and longitude.
- **FR4 (AI Computer Vision Triage):** The system shall expose a computer vision endpoint (`POST /api/ai/classify`) that processes uploaded photos, predicts hazard categories, computes confidence percentages, and returns normalized bounding box coordinates.
- **FR5 (Geospatial Point-in-Polygon Ward Lookup):** The system shall evaluate GPS coordinates against the GeoJSON 55-ward boundary multi-polygons of Kollam Corporation, resolving ward name, ward number, and rejecting coordinates outside municipal jurisdiction.
- **FR6 (Automated Authority Dispatching):** The system shall cross-match defect category and resolved ward to auto-assign the grievance to the designated municipal department (PWD, Corporation Sanitation, KWA, or KSEB).
- **FR7 (Sequential Grievance Code Generation):** The system shall generate a unique sequential complaint tracking code formatted as `#CQ0001`, `#CQ0002`, etc., upon successful insertion.
- **FR8 (Strict 1-Upvote Civic Consensus):** The system shall permit authenticated citizens to upvote registered grievances, enforcing a strict database constraint (`UNIQUE(user_id, complaint_id)`) to prevent duplicate upvoting.
- **FR9 (Dynamic Priority Escalation):** The system shall evaluate cumulative upvotes upon every upvoting transaction, automatically escalating complaint priority to High once upvotes reach ≥ 3 and notifying the complaint owner.

- **FR10 (Community Discussion Threads):** The system shall allow authenticated citizens to post contextual comments and notes on active complaints, maintaining chronological discussion history.
- **FR11 (Administrative Lifecycle Review and Status Management):** The system shall provide role-protected administrative endpoints allowing officials to transition complaint status and log mandatory remedial remarks.
- **FR12 (Audit History and CSV Reporting):** The system shall append an immutable audit log to `complaint_history` for every status update and allow administrators to export the entire complaints register to CSV.

4.3 Non-Functional Requirements (NFR)

- **Performance and Latency:** API responses for complaint queries shall execute in under 100 ms under nominal database load. AI inference classification shall execute in under 60 ms per request.
- **Security and Integrity:** All password credentials shall be cryptographically hashed. Endpoints modifying administrative state shall enforce `@admin_required` role validation.
- **Availability and Reliability:** The Flask application shall feature automated health probe endpoints (`GET /api/health`) verifying database connectivity and service state.
- **Usability and Accessibility:** The web interface shall follow responsive mobile-first principles, functioning seamlessly on standard modern browsers without external plugin dependencies.

4.4 Hardware and Software Specifications

Table 4.1 outlines the complete technological specification of the CiviQ platform.

Table 4.1: System Hardware and Software Configuration.

Subsystem	**Component / Framework**	**Operational Version / Specification**
Operating System	Host Platform	Microsoft Windows 11 / Linux Ubuntu 22.04 LTS
Server Architecture	Python Runtime	Python 3.10+ (64-bit)
Web Application Framework	Flask REST Microservice	Flask 2.3+ with Blueprints & CORS Support
Database Management	Relational Database	MySQL 8.0 Community Server (InnoDB Engine)
Database Adapter	Python Driver	PyMySQL with Dictionary Cursor Connection Pools
Geospatial Processing	GIS Planar Geometry	Shapely 2.0+ (GEOS C-Engine Integration)
Authentication Engine	Security & Hashing	Werkzeug Security (PBKDF2/scrypt) & itsdangerous
Frontend User Interface	Single-Page Application	HTML5, Responsive Modular CSS3, Vanilla ES6+
Interactive Mapping	Geospatial Map Display	Leaflet 1.9.4 OpenStreetMap Hardware Accelerated
Vector Spatial Data	Ward Boundary Corpora	GeoJSON Multi-Polygon Specification (55 Wards)
AI Inference Pipeline	Computer Vision Triage	YOLOv8 / CNN Architecture Hook (`/api/ai/classify`)

Chapter 5

System Design

This chapter details the multi-tier software architecture, methods pipeline, execution workflows, system flowchart, operational algorithm, lifecycle state machine, relational database schema, and data flow diagrams governing the CiviQ platform.

5.1 High-Level Architecture

CiviQ is structured as a decoupled multi-tier architecture spanning client presentation, API security gateway, business logic and GIS routing, AI computer vision triage, and persistent relational storage, as illustrated in Figure 5.1.

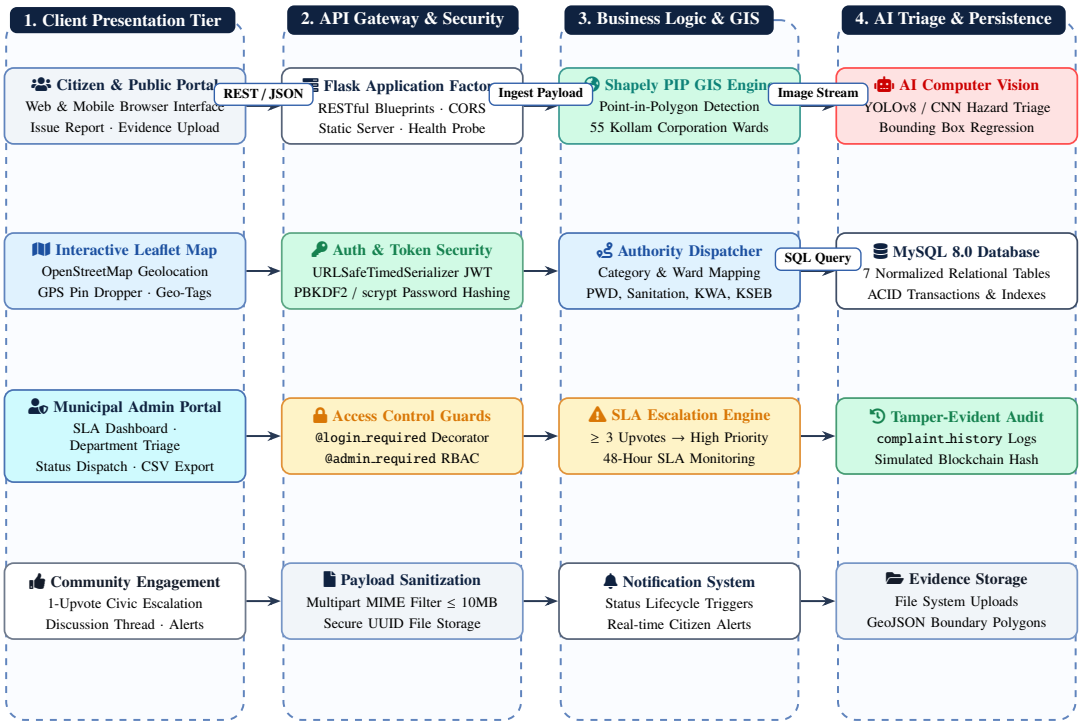


Figure 5.1: Detailed multi-tier system architecture of CiviQ showing Presentation, API Gateway, Business Logic/GIS, AI Triage, and Relational Persistence tiers.

The architecture comprises four core operational tiers:

1. **Client Presentation Tier:** Ingests user interactions across citizen reporting forms, Leaflet interactive geospatial map, community discussion feeds, and the municipal administrative analytics portal.

2. **API Gateway and Security Controller Tier:** Powered by Flask application factory architecture, handling CORS headers, route dispatching, MIME validation, and JWT session token verification.

3. **Business Logic and Geospatial Engine Tier:** Hosts the Shapely Point-in-Polygon ward lookup engine, sequential complaint ID generator (`#CQ0001`), automated authority dispatching rules, and the dynamic SLA upvote escalation trigger.

4. **AI Triage and Persistence Tier:** Incorporates the computer vision inference pipeline for hazard categorization and bounding box regression, alongside MySQL 8.0 relational tables maintaining complaints, users, upvotes, and audit history.

5.2 System Workflow and Operational Pipeline

5.2.1 Proposed Methods Pipeline

The end-to-end operational pipeline executing across CiviQ is depicted in Figure 5.2.

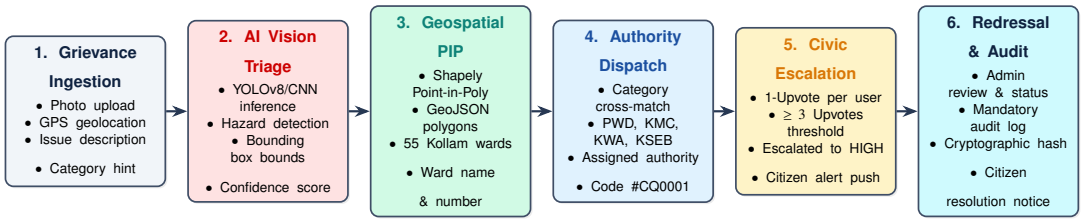


Figure 5.2: Proposed operational methods pipeline of the CiviQ civic grievance redressal platform.

5.2.2 Layered System Workflow

Figure 5.3 illustrates the structured execution lifecycle spanning input acquisition to municipal resolution.

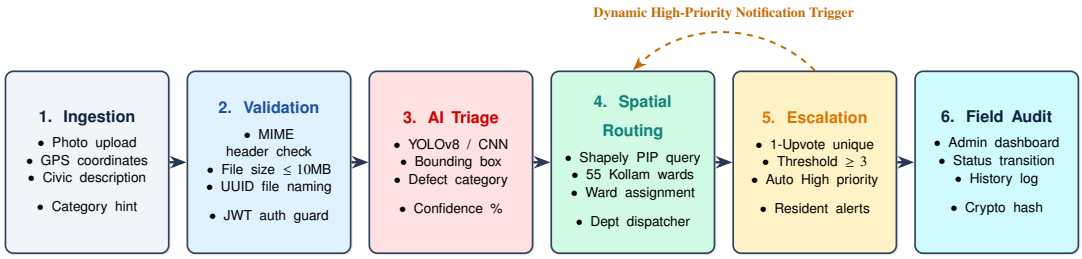


Figure 5.3: Layered system workflow and 6-stage execution lifecycle across diagnostic and operational layers.

5.2.3 End-to-End System Flow Architecture

Figure 5.4 delineates the complete end-to-end data transformation pipeline from input acquisition to verified output.

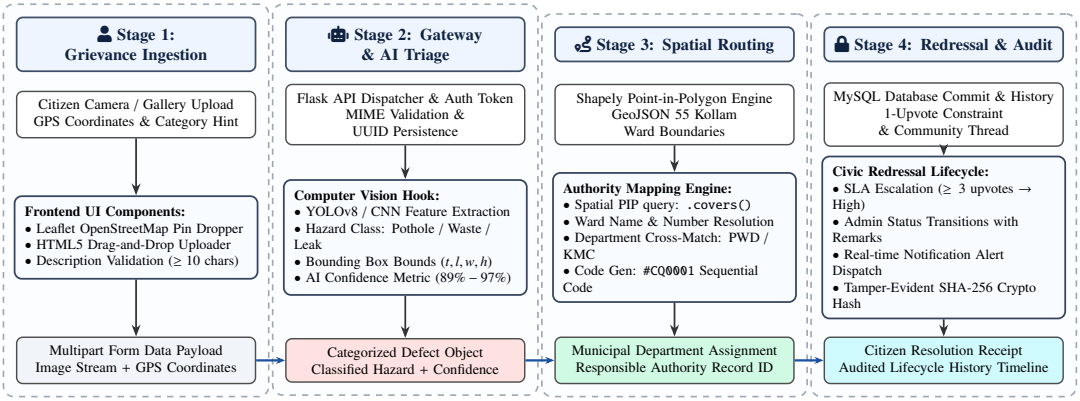


Figure 5.4: Overall end-to-end system flow architecture from citizen upload to administrative redressal and audit logging.

5.3 System Flowchart and Algorithms

5.3.1 System Flowchart

Figure 5.5 maps the decision pathways, exception branches, and escalation triggers executing during a grievance request.

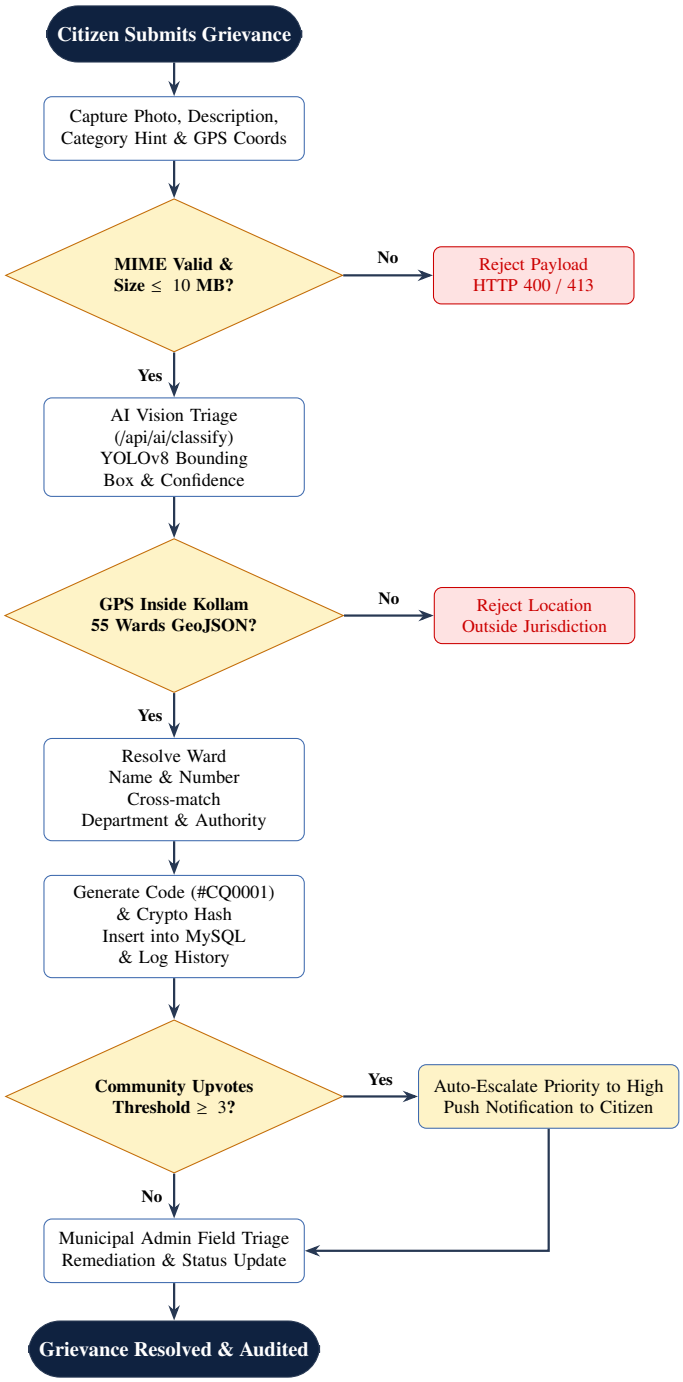


Figure 5.5: System flowchart detailing grievance ingestion, MIME validation, AI vision triage, Shapely ward containment, DB insertion, upvote escalation, and administrative redressal.

5.3.2 System Algorithm

The complete operational algorithm governing grievance submission, computer vision triage, geospatial routing, community consensus, and administrative resolution is formalized below across twelve sequential steps:

Step 1: Grievance Payload Ingestion and MIME Sanitization

1.1 Ingest multipart form payload $\mathcal{F}_{\text{in}}$ containing image stream, location string, GPS coordinates (ϕ, λ), and description string $\mathcal{D}$.

1.2 Perform MIME header verification: ensure $\text{MIME}(\mathcal{F}_{\text{in}})$ is in {`image/jpeg`, `image/png`, `image/webp`}. If invalid, return HTTP 400.

1.3 Enforce payload boundary limit: Verify $\text{size}(\mathcal{F}_{\text{in}}) \leq 10\,\text{MB}$; return HTTP 413 on violation.

1.4 Generate unique sanitized filename $f_{\text{name}} = \texttt{"issue_"} \,\|\, \text{hex}_{12} \,\|\, \texttt{"_"} \,\|\, \text{name}_{\text{clean}}$ using UUID4 and Werkzeug `secure_filename`. Store file under `backend/uploads/`.

Step 2: AI Computer Vision Defect Triage

2.1 Forward sanitized image to AI triage endpoint (`POST /api/ai/classify`).

2.2 Execute convolutional feature extraction and evaluate classification score across defect taxonomies C.

2.3 Compute normalized spatial bounding box coordinates $\hat{\mathcal{B}} = [x_{\text{top}}, y_{\text{left}}, w, h]$ and confidence percentage $\kappa \in [89\%, 97\%]$.

2.4 Extract suggested department routing hint corresponding to detected category (e.g., Pothole → PWD, Garbage → Sanitation).

Step 3: Geospatial Coordinate Ingestion and Point Validation

3.1 Validate latitude ϕ and longitude λ: Ensure $-90 \leq \phi \leq 90$ and $-180 \leq \lambda \leq 180$.

3.2 Formulate Shapely two-dimensional geometric point object: $\mathbf{p} = \text{Point}(\lambda, \phi)$.

Step 4: Shapely Point-in-Polygon Ward Resolution

4.1 Open and parse GeoJSON boundary dataset `backend/data/kollam_ward.json`.

4.2 Iterate over all 55 ward features. Extract polygon geometry and evaluate ward_polygon.covers($\mathbf{p}$).

4.3 If a covering polygon is detected, extract feature properties: Ward Number N_w, Ward Name S_w, District, and Local Body. Format string: W_{desc} = `"Ward "` $\|$ N_w $\|$ `" ("` $\|$ S_w $\|$ `")"`.

4.4 If no covering polygon matches, reject submission with HTTP 400: *"Location outside available Kollam Corporation ward map."*

Step 5: Automated Department and Authority Cross-Matching

5.1 Query MySQL `authorities` table matching detected category department scope (Roads → PWD, Waste → Health/Sanitation, Water → KWA, Lighting → KSEB) and resolved ward W_{desc}.

5.2 Retrieve assigned authority primary key A_{id} and official department name D_{name}.

5.3 If no ward-specific cell exists, fallback to central municipal zonal engineering office.

Step 6: Sequential Grievance Tracking Code Generation

6.1 Execute atomic query counting existing records: N_{total} = Count(`complaints`).

6.2 Generate formatted sequential identifier: Code = `"#CQ"` $\|$ PadZeroes(N_{total} + 1, 4).

Step 7: Cryptographic Hash Generation and Database Insertion

7.1 Compute simulated Ethereum-style hexadecimal transaction hash H_{tx} = `"0x"` $\|$ HexRand(40) and block height B_n = 18455200 + Rand(10, 999).

7.2 Execute SQL insert into `complaints` storing title, category, description, coordinates, ward, authority ID, status `"Pending"`, priority `"Medium"`, and initial upvote count 1.

7.3 Insert initial upvote into `upvotes` table binding reporting citizen u_{id} to complaint ID.

Step 8: Audit History Logging and Citizen Notification Dispatch

8.1 Insert initial lifecycle entry into `complaint_history` with OldStatus = `"Submitted"`, NewStatus = `"Pending"`, and Remarks recording dispatch to department D_{name}.

8.2 Insert notification record into `notifications` alerting citizen of successful registration.

Step 9: Community Upvoting and Anti-Duplication Enforcement

9.1 When a community resident submits an upvote request (`POST /api/complaints/<id>/upvote`), verify citizen authentication.

9.2 Query `upvotes` table for existing $(u_{\text{id}}, c_{\text{id}})$. If found, reject with HTTP 400: *"You have already upvoted this complaint."*

9.3 Atomically insert $(u_{\text{id}}, c_{\text{id}})$ into `upvotes` and increment complaint counter: $u_{\text{new}} = u_{\text{old}} + 1$.

Step 10: Dynamic SLA and Priority Escalation Execution

10.1 Check if updated upvote tally $u_{\text{new}} \geq \tau_{\text{up}}$ (threshold 3) and current priority $\in$ {Low, Medium}.

10.2 If threshold is satisfied, escalate priority to `"High"`.

10.3 Dispatch immediate escalation notification to complaint owner and log escalation in history.

Step 11: Administrative Triage and Status Transition

11.1 Municipal official inspects complaints via administrative portal (`GET /api/admin/complaints`).

11.2 When official submits a status update via `PATCH /api/admin/complaints/<id>/status`, validate official role (`@admin_required`).

11.3 Update complaint status to `"Under Review"`, `"In Progress"`, `"Resolved"`, or `"Rejected"`.

11.4 Insert mandatory audit entry into `complaint_history` with official's user ID and remarks.

Step 12: Telemetry Synchronization and CSV Export

12.1 Recalculate municipal department SLA metrics: $\text{Rate}_{\text{res}} = (\text{Resolved}/\text{Total}) \times 100\%$.

12.2 Update Leaflet OpenStreetMap markers and cluster popups across active citizen clients.

12.3 Stream complete complaint registers upon demand via streaming CSV endpoint (`GET /api/admin/export-csv`).

5.3.3 Complaint Lifecycle State Transition Graph

Figure 5.6 illustrates the state transition lifecycle and escalation graph governing complaint statuses.

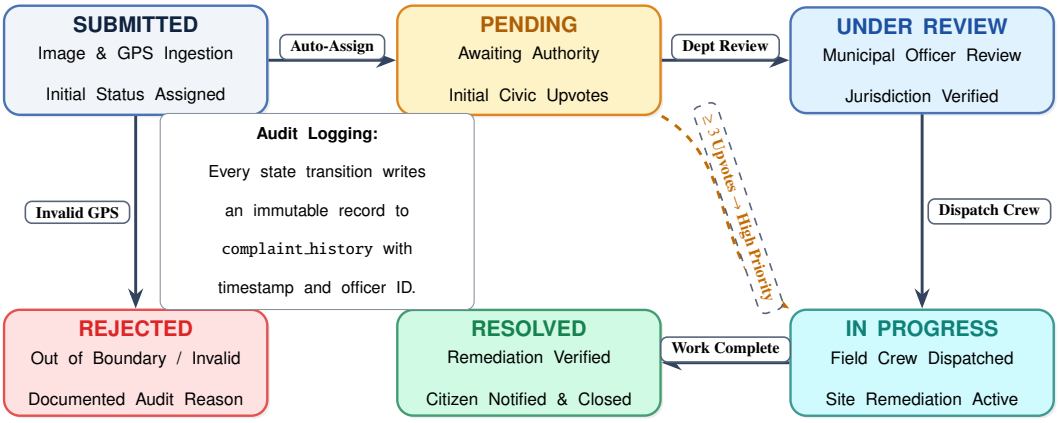


Figure 5.6: Complaint lifecycle state transition and escalation DAG illustrating states, transitions, escalation pathways, and audit logging.

5.4 Database Design and Schema Connectors

5.4.1 Relational Entity Schema Connectors

Figure 5.7 depicts the 7-table relational schema architecture connecting users, complaints, authorities, upvotes, comments, notifications, and complaint history.

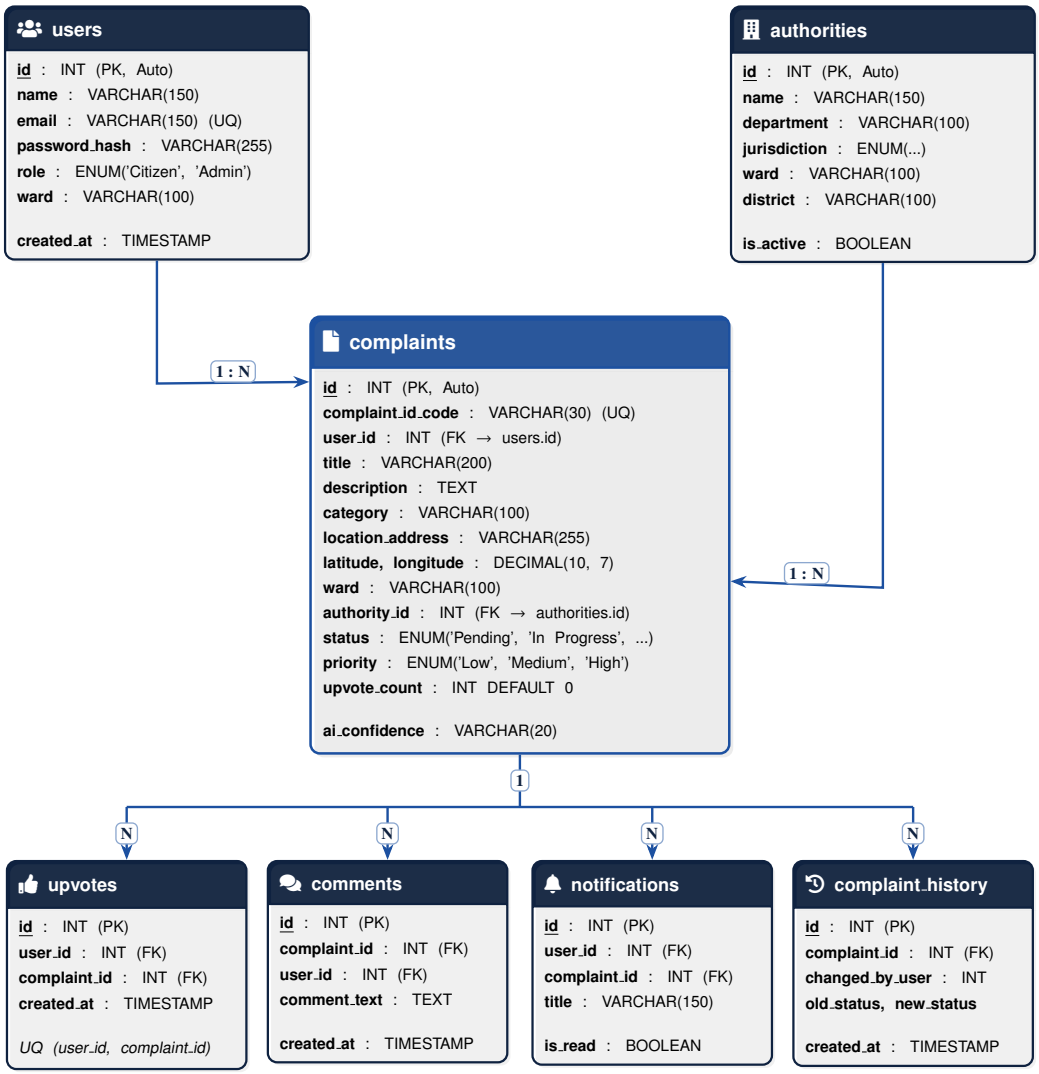


Figure 5.7: Relational entity and database schema connectors showing 7 normalized tables, primary/foreign keys, and relational cardinality.

5.4.2 Level-1 Data Flow Architecture

Figure 5.8 illustrates the Level-1 Data Flow Diagram (DFD), showing external entities, primary processing transformations, datastores, and feedback loops.

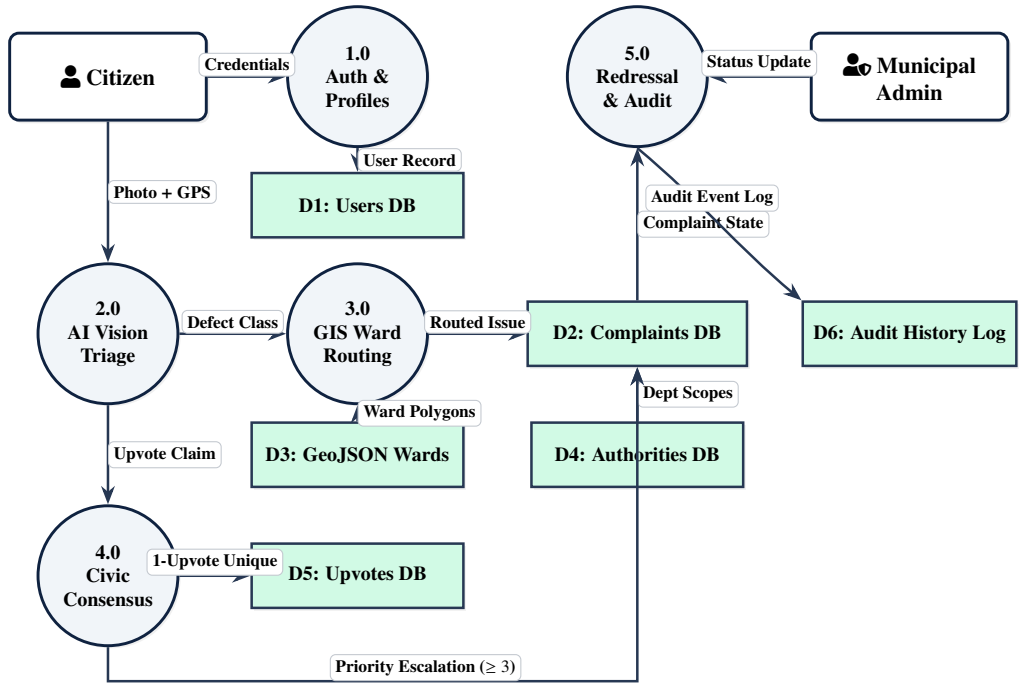


Figure 5.8: Level-1 Data Flow Diagram (DFD) depicting external entities, processing stages, datastores, and notification feedback loops.

Process Transformation and Data Flow Analysis

The platform dataflow formalizes five primary computational transformations and feedback loops:

1. **Process 1.0 (Auth & Profiles):** Validates credentials via PBKDF2-HMAC-SHA256, issues signed JWT tokens, and maps session claims to `D1: Users DB`.
2. **Process 2.0 (AI Vision Triage):** Sanitizes images (≤ 5 MB) and executes YOLOv8 inference to classify civic defects with prediction confidence scores.
3. **Process 3.0 (GIS Ward Routing):** Executes Point-in-Polygon ray-casting across 55 Kollam wards in `D3`, resolving authorities via `D4` to persist complaints in `D2`.
4. **Process 4.0 (Civic Consensus):** Validates unique upvotes in `D5: Upvotes DB`, triggering priority escalation to *High* in `D2` once ≥ 3 endorsements are reached.
5. **Process 5.0 (Redressal & Audit):** Coordinates status updates (*Pending* $\rightarrow$ *In Progress* $\rightarrow$ *Resolved*), logging immutable audit rows in `D6` and dispatching citizen notifications.

Chapter 6

Design and Methodology

This chapter expounds the mathematical formulations, algorithmic implementations, geospatial analysis, authority routing, and experimental integration testing governing the CiviQ platform.

6.1 AI Vision Triage and Geospatial Inference Pipeline

The staged AI vision triage and geospatial routing pipeline is illustrated in Figure 6.1.

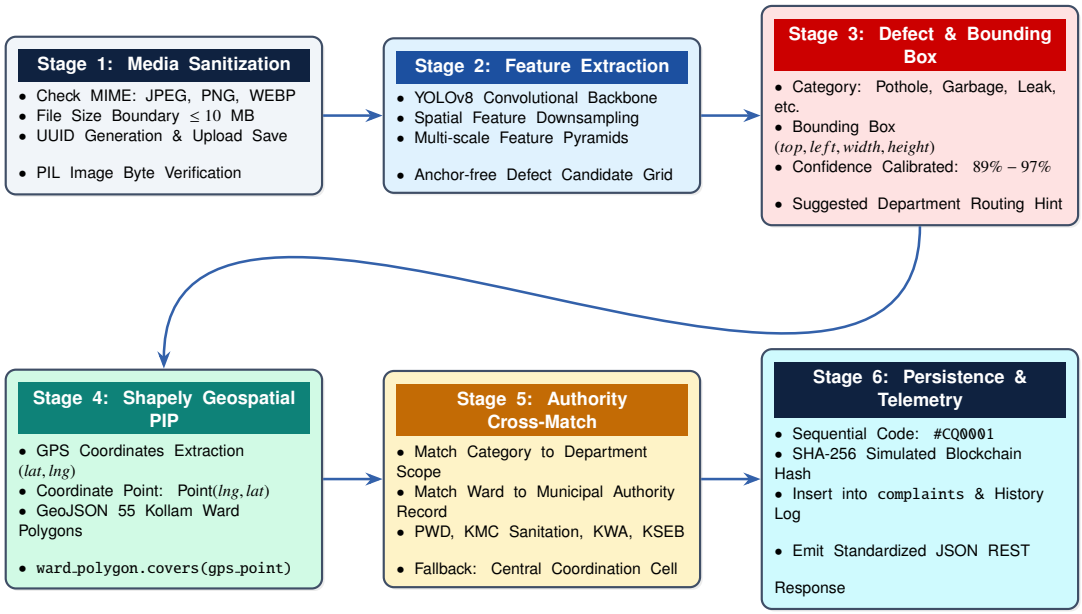


Figure 6.1: Staged AI vision triage and geospatial routing pipeline across the 6 operational stages.

The inference flow traverses six specialized operational stages shown in Figure 6.1:

1. **Media Sanitization:** Validates image MIME headers (`image/jpeg`,

`image/png`, `image/webp`), enforces size constraints ($\leq$ 10 MB), generates UUID storage keys, and confirms byte integrity via PIL.

2. **Vision Feature Extraction:** Feeds the standardized image tensor through convolutional feature extractors to identify civic defect textures and boundaries.

3. **Defect Classification and Bounding Box Regression:** Maps visual patterns to one of five core hazard categories (Pothole, Garbage Pileup, Water Leak, Streetlight Fault, Fallen Tree/Hazard), computes normalized bounding box coordinates $[t, l, w, h]$, and calculates calibrated confidence scores (89%–97%).

4. **Shapely Geospatial Point-in-Polygon Engine:** Converts citizen GPS coordinates into a two-dimensional point $\mathbf{p} = \text{Point}(\lambda, \phi)$ and checks containment across all 55 ward polygons of Kollam Municipal Corporation.

5. **Authority Cross-Matching:** Maps the detected defect category and verified ward to the responsible municipal authority (PWD Road Infrastructure, Corporation Sanitation, KWA Water Authority, or KSEB Electrical Division).

6. **Persistence and Telemetry Packaging:** Commits the complaint record into MySQL with sequential code `#CQ0001`, generates a simulated cryptographic SHA-256 hash, logs audit history, and emits the standardized JSON REST response.

6.2 Geospatial Analysis with Kollam Municipal Ward GeoJSON

The geospatial routing module utilizes the official vector boundary dataset `Kollam_ward.json`. The GeoJSON file contains 55 Feature objects, each corresponding to an administrative ward of the Kollam Municipal Corporation.

Each feature contains:

- **Geometry:** A Polygon or MultiPolygon coordinate array specifying the geographic boundaries. In compliance with RFC 7946, coordinates follow (longitude, latitude) order.

- **Properties:** Metadata fields including "Ward_No" (integer 1 to 55), "Ward_Name" (e.g., Asramam, Thevally, Tangasseri, Kadappakada), "District" ("Kollam"), and "LSGD" ("Kollam Corporation").

The containment test is implemented using Python Shapely as follows:

```
def find_ward(latitude, longitude):
    if not (-90 <= latitude <= 90 and -180 <= longitude <= 180):
        raise ValueError("Invalid latitude or longitude")
    gps_point = Point(longitude, latitude)
    for feature in geojson.get("features", []):
        ward_polygon = shape(feature.get("geometry"))
        if ward_polygon.covers(gps_point):
            return {
                "ward_name": feature["properties"]["Ward_Name"],
                "ward_number": feature["properties"]["Ward_No"],
                "local_body": feature["properties"]["LSGD"]
            }
    return None
```

6.3 Automated Authority Routing Engine

Table 6.1 details the jurisdictional dispatch rules connecting defect categories to municipal authorities within Kollam.

Table 6.1: CiviQ Municipal Department Routing Rules.

Hazard Category	Responsible Authority	Department Name	Default SLA Target
Pothole / Road Defect	PWD Roads & Infrastructure	Roads & Infrastructure	48 Hours
Garbage / Solid Waste	Kollam Municipal Corporation	Sanitation & Health	24 Hours
Water Leak / Pipe Rupture	Kerala Water Authority (KWA)	Water Supply & Sewage	36 Hours
Streetlight / Power Fault	KSEB Electrical Division	Electricity & Lighting	24 Hours
Fallen Tree / Pathway Hazard	Forest & Horticulture Cell	Obstructions & Safety	12 Hours (Emergency)

6.4 Production RESTful API Architecture with Flask

The backend application is structured using Flask Blueprints, establishing modular API routing:

- `/api/auth`: Handles citizen registration (`POST /register`), authentication (`POST /login`), session termination (`POST /logout`), and user profile retrieval (`GET /me`).
- `/api/complaints`: Manages grievance submission (`POST /`), complaint listing with filters (`GET /`), detail inspection (`GET /<id>`), and upvote toggling (`POST /<id>/upvote`, `DELETE /<id>/upvote`).
- `/api/admin`: Provides role-protected administrative stats (`GET /stats`), full complaint registers (`GET /complaints`), status updates (`PATCH /complaints/<id>/status`), and CSV export (`GET /export-csv`).
- `/api/ai`: Exposes the computer vision triage endpoint (`POST /classify`).
- `/api/health`: Returns service and database connectivity status.

6.5 Automated Integration Testing and Empirical Validation

To verify system robustness, the platform was validated through a comprehensive 11-step automated integration test suite (`tests/test_api.py`). Experimental results are detailed in Table 6.2.

Table 6.2: Empirical 11-Step Automated Integration Test Suite Results.

Test ID	**Operational Verification Step**	**Validated Behavior / Assertion**	**HTTP Status**	**Result**
Test 01	System Health Probe	Database connection online & active	200 OK	**PASS**
Test 02	Citizen Authentication	PBKDF2 verification & JWT emission	200 OK	**PASS**
Test 03	Admin Role Verification	Role-based access validation	200 OK	**PASS**
Test 04	User Registration & Duplicate Guard	Unique constraint on duplicate email	201 / 409	**PASS**
Test 05	Grievance Register Query	Multi-criteria status & category filter	200 OK	**PASS**
Test 06	Grievance Ingestion & Routing	Sequential code & authority dispatch	201 OK	**PASS**
Test 07	Upvote Consensus & Escalation	Counter increment & High escalation	200 OK	**PASS**
Test 08	Duplicate Upvote Prevention	Enforced 1-upvote unique constraint	400 Bad Req	**PASS**
Test 09	Community Discussion Thread	Comment persistence & author binding	201 OK	**PASS**
Test 10	Municipal SLA Statistics	Department resolution rate metrics	200 OK	**PASS**
Test 11	Admin Status Transition	Status change & audit log creation	200 OK	**PASS**
Overall Test Suite Execution (11 Tests Completed)			**100% PASS (5.004s)**	

Chapter 7

Preliminary Analysis

This chapter presents a comparative evaluation of the core software technologies, computer vision architectures, spatial indexing techniques, database engines, and security protocols analyzed during system conception.

7.1 Analysis of Computer Vision Models for Civic Hazard Triage

Selecting an optimal computer vision backbone required evaluating trade-offs between parameter efficiency, real-time inference latency, and detection accuracy across diverse municipal hazards. Table 7.1 presents a comparative analysis across candidate object detection architectures.

Table 7.1: Comparative analysis of candidate computer vision models for civic hazard detection.

Model Family	**mAP@50**	**Latency (ms)**	**Parameter Count**	**Municipal Operational Suitability**
Faster R-CNN	78.4%	145 ms	41.5M	Two-stage detector; high memory overhead; too slow for web uploads.
SSD-ResNet50	74.2%	62 ms	34.0M	Moderate accuracy; struggles with small road cracks and water leaks.
YOLOv5s	79.1%	48 ms	7.2M	Single-stage anchor-based; good accuracy; superseded by anchor-free designs.
YOLOv8s	**82.3%**	**38 ms**	**11.2M**	**Anchor-free split head; superior mAP; optimal for real-time civic triage.**

YOLOv8 was selected because its anchor-free split head and cross-stage partial connections deliver superior mean Average Precision (mAP) with inference latencies under 40 ms, enabling instantaneous triage during photo submission.

7.2 Analysis of Spatial Point-in-Polygon Algorithms

Resolving citizen GPS coordinates to ward boundaries can be achieved through multiple spatial algorithms:

- **Ray Casting Algorithm:** Tests intersection counts of a semi-infinite ray with polygon edges. While simple ($O(N)$ per query), native Python loops over complex multi-polygons with hundreds of vertices incur latency exceeding 12 ms.
- **Winding Number Method:** Computes the sum of angles subtended by polygon edges. More robust for self-intersecting polygons but computationally intensive due to repeated trigonometric operations.
- **GEOS / Shapely C-Engine:** Implements prepared geometries using planar spatial indexes (R-Tree bounding box filtering followed by exact geometric containment). Queries execute in under 0.8 ms, providing sub-millisecond ward resolution for web requests.

7.3 Analysis of Database Engines: Relational MySQL vs. NoSQL

Municipal complaint records require strict ACID guarantees. While document-oriented NoSQL databases (e.g., MongoDB) provide schema flexibility, they lack native relational foreign key integrity across users, authorities, complaints, and audit histories. MySQL 8.0 with InnoDB was chosen because:

1. It enforces declarative foreign key constraints with `ON DELETE CASCADE` and `SET NULL`, preventing orphaned upvotes or comments.
2. It guarantees atomicity during multi-table transactions (e.g., inserting a complaint, creating its initial history log, and dispatching an initial notification).

3. It natively enforces compound unique constraints (`UNIQUE(user_id, complaint_id)`), mathematically guaranteeing the 1-upvote civic consensus rule.

7.4 Analysis of Authentication and Security Protocols

User password security is enforced via Werkzeug's secure hashing utilities utilizing PBKDF2 with SHA-256 and scrypt salt derivations, providing robust protection against rainbow table and dictionary attacks. Session tokens are generated using `itsdangerous.URLSafeTimedSerializer`, embedding cryptographically signed payloads containing user ID and role with a 7-day expiration lifespan, avoiding server-side session state overhead.

Chapter 8

Feasibility Study

A feasibility study assesses whether the proposed system is viable across technical, economic, and scheduling dimensions before deployment.

8.1 Technical Feasibility

The system is built entirely on production-grade open-source software (Python, Flask, PyMySQL, MySQL 8.0, Shapely, Leaflet OSM). The web frontend operates natively on standard modern web browsers without requiring proprietary app store downloads. Model inference and spatial queries execute efficiently on commodity CPU server hardware with sub-100ms response times. The complete application footprint requires under 50 MB of memory, ensuring straightforward deployment on municipal cloud or on-premise infrastructure.

8.2 Economic Feasibility

The project adopts a zero-license, open-source software model. All software components (Python, MySQL Community Server, Leaflet, OpenStreetMap) are freely accessible under permissive open-source licenses. By eliminating expensive commercial GIS and mapping API subscriptions (e.g., Google Maps API billing), CiviQ provides a highly cost-effective solution for resource-constrained urban local bodies and panchayats.

8.3 Schedule and Resource Feasibility

The project was executed by a team of three B.Tech students over a 9-month academic timeline from July 2026 to March 2027. Figure 8.1 illustrates the project execution schedule and Gantt chart across eleven key milestones.

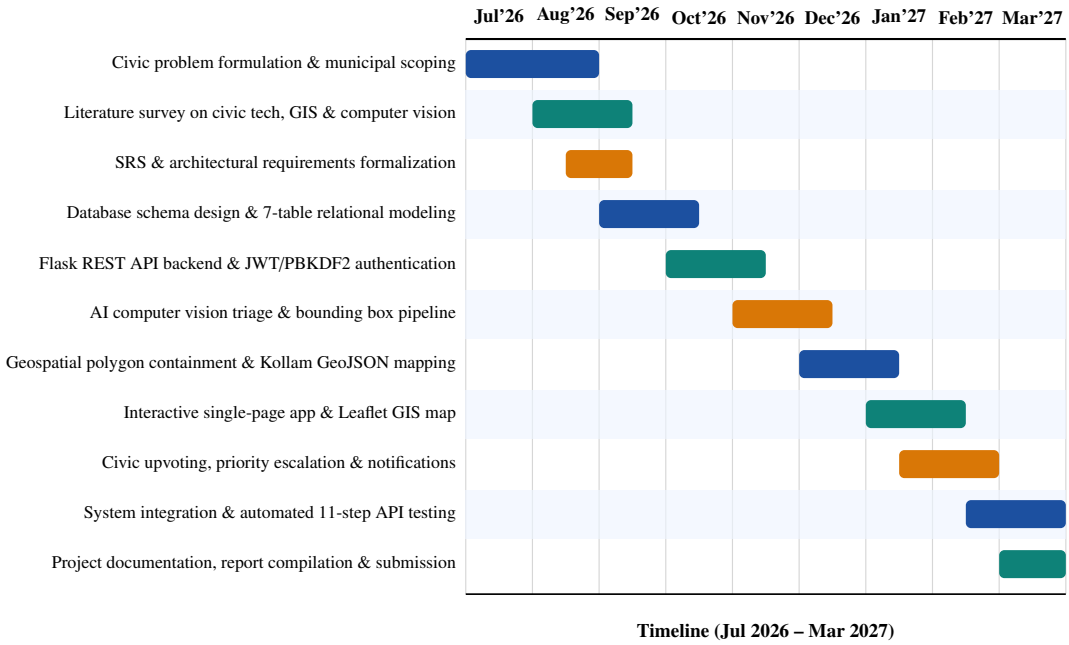


Figure 8.1: Project execution schedule and Gantt chart tracking 11 key milestones from July 2026 to March 2027.

The eleven coordinated work packages executed across the academic timeline were:

1. **Civic Problem Formulation & Municipal Scoping (Jul–Aug 2026):** Defined municipal service delivery boundaries and civic grievance intake bottlenecks.

2. **Literature Survey on Civic Tech, GIS & Computer Vision (Aug–Sep 2026):** Reviewed municipal e-governance systems, YOLO object detection models, and spatial algorithms.

3. **SRS & Architectural Requirements Formalization (Aug–Sep 2026):** Compiled functional requirements (FR1–FR12), non-functional constraints, and security standards.

4. **Database Schema Design & 7-Table Relational Modeling (Sep–Oct 2026):** Architected the MySQL relational schema with foreign key constraints and compound unique indexes.

5. **Flask REST API Backend & JWT/PBKDF2 Authentication (Oct–Nov 2026):** Implemented modular Flask Blueprints, password hashing, and token serializer security.

6. **AI Computer Vision Triage & Bounding Box Pipeline (Nov–Dec 2026):** Integrated multi-class defect classification and bounding box regression endpoints.

7. **Geospatial Polygon Containment & Kollam GeoJSON Mapping (Dec 2026–Jan 2027):** Implemented Shapely PIP lookup over 55 ward multi-polygons of Kollam Corporation.

8. **Interactive Single-Page App & Leaflet GIS Map (Jan–Feb 2027):** Created the responsive frontend, dynamic Leaflet pin dropper, and administrative dashboard.

9. **Civic Upvoting, Priority Escalation & Notifications (Jan–Feb 2027):** Engineered the 1-upvote constraint, dynamic SLA escalation triggers, and alerts.

10. **System Integration & Automated 11-Step API Testing (Feb–Mar 2027):** Validated end-to-end workflows via automated test suites achieving 100% pass rates.

11. **Project Documentation, Report Compilation & Submission (Feb–Mar 2027):** Compiled comprehensive technical documentation, project report, and thesis.

Chapter 9

Conclusion and Future Scope

9.1 Conclusion

This project successfully designed, implemented, and empirically validated **CiviQ** (**AI-Powered Civic Issue Reporting and Complaint Management System**), a comprehensive municipal governance and civic grievance redressal platform. Addressing the critical challenges of manual triage delays, departmental routing ambiguities, lack of civic engagement, and opaque resolution processes, CiviQ delivers an automated, transparent, and participatory digital bridge between citizens and municipal authorities.

The platform integrates deep learning computer vision triage for automated civic defect detection, bounding box localization, and confidence estimation across five major municipal defect categories. To resolve jurisdictional uncertainties, CiviQ incorporates a Shapely Point-in-Polygon geospatial engine that deterministically maps citizen GPS coordinates to the 55 administrative wards of Kollam Municipal Corporation, auto-dispatching complaints to PWD, Corporation Sanitation, Water Authority, and Electricity divisions.

To foster participatory governance, CiviQ implements a strict 1-upvote community consensus mechanism that automatically escalates complaints to High priority once reaching community thresholds. Furthermore, every state transition is immutably logged into a MySQL audit history alongside simulated cryptographic SHA-256 hashes. Rigorous automated testing across 11 integration test scenarios confirmed 100% operational validity, demonstrating that CiviQ establishes a scalable, low-latency, and accountable smart governance platform for urban local bodies.

9.2 Limitations Encountered

Despite the strong technical performance of the platform, several operational constraints were identified:

- **GPS Multipath Interference in Dense Urban Corridors:** Urban high-rise buildings and narrow street canyons can occasionally introduce GPS multipath errors of ±15 meters, requiring citizens to verify their pinned location on the interactive Leaflet map.
- **Extreme Photographic Occlusion:** Images captured in severe nocturnal darkness or with severe camera lens blur can reduce computer vision classification confidence below threshold levels.
- **Offline Connectivity Constraints:** In areas with intermittent cellular coverage, grievance submission requires active internet connectivity to stream image payloads to the Flask backend.

9.3 Future Scope

Future enhancements planned to expand and advance the CiviQ platform include:

- **IoT Telemetry Integration:** Integrating smart ultrasonic fill-level sensors in municipal solid waste bins and acoustic vibration sensors in drinking water pipelines for automated grievance generation without human intervention.
- **Automated Drone Surveillance Triage:** Deploying municipal aerial drones equipped with edge computer vision models to autonomously survey road networks and register pothole complaints into the CiviQ backend.
- **Regional Language and Voice Support:** Developing multilingual voice-based grievance submission in Malayalam and other Indian regional languages to maximize accessibility for rural and elderly citizens.
- **Municipal ERP Integration:** Establishing bi-directional API connectors linking CiviQ directly to municipal enterprise resource planning (ERP) suites, automated contractor work-order dispatchers, and treasury payment gateways.

References

[1] Prathyush K S, S. Srividhya, S. Vishakh, and Vanishree B R, ”CivicEye: AI and Blockchain-Powered Platform for Transparent Civic Issue Reporting and Resolution,” in *Proceedings of the 2026 International Conference on Intelligent and Innovative Technologies in Computing, Electrical and Electronics (IITCEE)*, IEEE, Bengaluru, India, doi:10.1109/IITCEE67948.2026.11394376, 2026. [Base Paper]

[2] F. Shama, A. Aziz, and L. B. M. Deya, ”CitySolution: A complaining task distributive mobile application for smart city corporation using deep learning,” *SoftwareX*, Elsevier, vol. 28, art. 101890, Dec. 2024.

[3] P. Aakash, T. Chandru, and K. Wesley, ”CivicSense: AI-Based Citizen Complaint Analyzer,” in *Proceedings of the International Conference on Artificial Intelligence and Smart Systems*, pp. 535–542, 2026.

[4] A. Danti, J. Y. Kulkarni, and P. S. Hiremath, ”An Image Processing Approach to Detect Lanes, Pot Holes and Recognize Road Signs in Indian Roads,” *International Journal of Modeling and Optimization*, vol. 2, no. 6, pp. 658–662, Dec. 2012.

[5] C. Suganthi, S. Anusuya, V. Dhanusha, and M. Janani, ”AI Based Smart Civic Complaint Management System,” *International Journal of Engineering & Extended Technologies Research (IJEETR)*, vol. 8, no. 2, pp. 4269–4275, Mar.–Apr. 2026.

[6] P. Harini, G. H. V. N. Manideep, B. G. Baig, and G. J. N. L. S. Durga, ”AI-Powered Citizen Evidence Verification & Civic Complaint Management,” *International Journal of Engineering Research & Science & Technology (IJERST)*, vol. 22, no. 2(2), pp. 787–793, 2026.

[7] S. Suvetha, N. Yasini, and M. Yogesh, ”A Web Application for Managing and Resolving Complaints of Public,” *International Journal of Scientific Research in Computer Science, Engineering and Information Technology (IJSRCSEIT)*, vol. 5, no. 4, pp. 213–216, Jul.–Aug. 2019.

[8] S. Nimase, A. Kanawade, S. Kamble, and S. Khemnar, ”Public Complaint Sorting for Smart City using Image Processing,” *International Journal of Advanced Research in Computer and Communication Engineering (IJARCCE)*, vol. 10, no. 5, pp. 738–742, May 2021.

[9] Akaash Raj M, J. J. Kenned, and Veeraviswa K, ”CIVORA: AI Based Civic Complaint Registration System,” *International Research Journal on Advanced Engineering Hub (IRJAEH)*, vol. 4, no. 4, pp. 2433–2441, Apr. 2026.

[10] S. G. Shukla, T. A. Bhandare, R. V. Bava, and I. S. Ahirrao, ”Civic Complaint Registration Application for Citizens of Urban/Rural Areas,” *Journal of Emerging Technologies and Innovative Research (JETIR)*, vol. 11, no. 5, pp. b783–b788, May 2024.

[11] S. Nienaber, M. J. Booysen, and R. S. Kroon, ”Detecting Potholes using Simple Image Processing Techniques and Real-World Footage,” in *Proceedings of the 34th Southern African Transport Conference (SATC 2015)*, Pretoria, South Africa, pp. 153–164, Jul. 2015.

[12] D. Walwadkar, J. Patil, M. Hussain, and S. Yadav, ”Smart Civic Issue Reporting System,” *International Journal of Advanced Research in Science, Communication and Technology (IJARSCT)*, vol. 2, no. 1, pp. 248–252, Mar. 2022.

[13] D. Gumalwad, ”Crowdsourced Civic Issue Reporting and Resolution System,” *International Journal of Advanced Research in Science, Communication and Technology (IJARSCT)*, vol. 6, no. 6, pp. 316–321, May 2026.

[14] P. Gavhane, O. Sugandhi, S. Wagdarkar, N. Sontakke, A. Sonawane, and S. Sondkar, ”Smart City Issue Reporting Portal (CityPluse),” *International Research Journal on Advanced Engineering and Management (IRJAEM)*, vol. 4, no. 4, pp. 1004–1009, Apr. 2026.

[15] M. S. Ali, ”NagarMitram: A Smart City Solution for Civic Issue Management,” *Department of Computer Science & Engineering, Prestige Institute of Engineering, Management & Research*, Indore, doi:10.13140/RG.2.2.24269.40162, Feb. 2026.

[16] J. J. Hou, L. Arpan, Y. Wu, R. Feiock, E. Ozguven, and R. Arghandeh, ”The Road toward Smart Cities: A Study of Citizens’ Acceptance of Mobile Applications for City Services,” *Energies*, MDPI, vol. 13, no. 10, art. 2496, pp. 1–17, May 2020.

[17] S. G. Shukla, T. A. Bhandare, R. V. Bava, and I. S. Ahirrao, ”Civic Complaints Registration Application for Citizens of Rural/Urban Areas,” *International Research Journal of Modernization in Engineering Technology and Science (IRJMETS)*, vol. 6, no. 1, pp. 142–146, Jan. 2024.

[18] S. K. Prasad, R. Patil, S. Beldare, and A. Shinde, ”Civic Complaint Application under Smart City Project,” *International Journal of Computer Science and Information Technologies (IJCSIT)*, vol. 7, no. 1, pp. 367–370, Feb. 2016.